8

Adding Windows Notifications to WinUI Applications

The Windows App SDK provides developers with the ability to implement **raw push notifications** and **app notifications** in their WinUI apps. It's important to understand the use case for each of these notification types. They have different implementations, and each has its own set of advantages and limitations. Push notifications can be surfaced to the user or received by the app to perform internal operations. App notifications, on the other hand, are used to communicate with the user. We'll cover examples of when you would want to use a particular notification type and add an app notification to the **My Media Collection** sample application.

In this chapter, we will cover the following topics:

- Understanding the different notification types in the Windows App SDK and the use case for each type
- Discovering how to leverage push notifications in a WinUI application
- Exploring how to use app notifications with WinUI

By the end of this chapter, you will understand the differences between push notifications and other app notifications exposed by the Windows App SDK. You will understand when to choose each notification type and how they are handled in a WinUI 3 project.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10 version 1809 (build 17763) or newer
- Visual Studio 2022 or later with the .NET Desktop Development workload configured for Windows App SDK development

The source code for this chapter is available on GitHub at this URL: **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter08**.

Overview of push notifications in the Windows App SDK

WinUI applications can leverage different types of notifications in the Windows App SDK. The notifications APIs were added in Windows App SDK 1.3 and can be either sent locally or through a cloud service, depending on the notification type. We most often associate notifications with the small, pop-up windows in the corner of the screen, called **toast notifications**, in Windows. However, a visual indicator isn't required for all notifications. They can also be used to signal your app to activate and perform an action or sync data from a remote service without relying on a timer in the app.

Raw push notifications

These internal notifications are known as **raw push notifications**. They require no user interaction and don't signal the user with a **toast notification**. Push notifications leverage the **Windows Push Notification Services (WNS)**, which is part of the **Microsoft Store** services. To publish an application in the Store or leverage any of its services, a Store account is required, and your app must be registered in the Store's dashboard.

NOTE

*We will discuss publishing apps to the Microsoft Store in **Chapter 14**, Packaging and Deploying WinUI Applications. We won't cover the Store registration process in this chapter, but you can skip ahead to review the process in **Chapter 14** if it's new to you.*

Push notifications from WNS can be received directly by the app to signal the app to perform some action. In fact, your app doesn't need to be active to receive a notification. Windows will activate the app so it can process the notification and perform the requested action. Using notifications saves device resources and can reduce or eliminate the need for polling and timers.

Notifications from WNS may also notify the user. This is one type of app notification.

Cloud-based app notifications

App notifications involve notifying the user that some event has occurred or action is required. App notifications can be local or originate from the cloud. The cloud-based notifications, like raw notifications, leverage WNS.

The process for creating and sending these app notifications is similar to creating raw push notifications. The header and content types will distinguish the app notifications and signal Windows to display a visible, transient notification. Any notifications that haven't been dismissed or cleared by the user can be viewed in Windows settings in **Notification Center**.

NOTE

Certain types of self-contained apps or apps running with admin privileges may not be eligible to receive notifications. To view more information about

these limitations, you can review this section of the push notifications documentation on Microsoft Learn: **https://learn.microsoft.com/windows/apps/windows-app-sdk/notifications/push-notifications/#limitations**.

App notifications can also be local to the user's PC. Let's discuss this type of notification next.

Local app notifications

Local app notifications do not involve the cloud and WNS is not involved in sending the notification. They originate from your app, are displayed to the user, and are handled by your app when the user acts on the toast notification. Users are familiar with these types of notifications from using Microsoft apps such as Outlook, Teams, and even the Microsoft Store app.

Sometimes, these notifications are informational, such as when the Store app displays a message after an app has been updated. The notifications can also prompt the user to take an action, such as snoozing an Outlook calendar reminder. In this case, the notification window contains a drop-down control that allows the user to select the snooze duration.

Later in this chapter, we'll add a local app notification to the **My Media Collection** app to prompt our users to add a new book to their collection. Now, we'll dive a little deeper into the implementation of raw push notifications and how they can be used to quietly receive a notification from the cloud.

Using raw push notifications in WinUI applications

As we discussed in the previous section, push notifications that are handled by the app without notifying the user are generated through WNS and Azure. In this section, we will briefly examine how these notifications can be leveraged in WinUI applications. The Azure configuration needed to get started is somewhat lengthy and not very interesting. Because the Azure Notification Hubs configuration for WNS is already well documented in the Azure docs on Microsoft Learn, you should review them before we get started: **https://learn.microsoft.com/azure/notification-hubs/notification-hubs-windows-store-dotnet-get-started-wns-push-notification**. It's also a good idea to familiarize yourself with the WNS overview in the Windows design documentation on Microsoft Learn: **https://learn.microsoft.com/windows/apps/design/shell/tiles-and-notifications/windows-push-notification-services--wns--overview**.

NOTE

The Azure documentation was written for UWP apps, but the configuration instructions work just as well for a WinUI 3 application.

Once the Azure configuration is complete, the steps to use notifications in a WinUI 3 app are similar to UWP but not exactly the same. For a detailed example of working with push notifications from the cloud, you can read this Microsoft Learn article: **https://learn.microsoft.com/windows/apps/windows-app-sdk/notifications/push-notifications/push-quickstart**. In this chapter, we are focusing on app notifications, and will add those to our sample app in the next section. The high-level steps you will need to complete are as follows:

1. Add the COM activation information to your **`Package.appxmanifest`** file. Here's an example:

```
<Extensions>
  <!--Register COM activator-->
  <com:Extension Category="windows.comServer">
    <com:ComServer>
      <com:ExeServer Executable="MyApp\MyApp.exe" DisplayName="My App" Arguments="----WindowsAp
        <com:Class Id="[Azure AppId for App]" DisplayName="WinUI Push Notify" />
      </com:ExeServer>
    </com:ComServer>
  </com:Extension>
</Extensions>
```

2. Register with **`PushNotificationManager`** in the **`Microsoft.Windows.Push Notifications`** namespace and subscribe to **`PushNotificationChannel`** for the notification type.
3. Add code to the **`App`** class to check whether the application was launched or activated from the background as a result of a push notification.
4. Create a WNS channel and register that channel with the WNS service. These are the HTTP endpoints that will receive the notification data to be pushed to your app.
5. Use a tool such as **Postman** (**https://postman.com/**) to send an HTTP **`POST`** request with the push notification data. You will need to get an access token for the request with your Azure tenant ID, app ID, and client secret. For more information, see this page: **https://learn.microsoft.com/azure/active-directory/develop/howto-create-service-principal-portal#get-tenant-and-app-id-values-for-signing-in**.

Those are the basic steps, but there's much more to be learned. Make sure you read all of the articles linked in this section to learn about the nuances of using raw push notifications in a WinUI application.

Now, let's learn more about app notifications and adding send-and-receive capabilities to our sample application.

Adding Windows app notifications with the Windows App SDK

In this section, we're going to add some local app notifications to the **My Media Collection** project. The code we'll add to the project is based on

the Windows App SDK local app notification sample app created by the Microsoft Learn team. You can download the code for that project on GitHub: **https://github.com/microsoft/WindowsAppSDK-Samples/tree/main/Samples/Notifications/App/CsUnpackagedAppNotifications**.

We are adding two buttons to the MainPage in the app to trigger two types of notifications. One will have an image and some text. The second will add a text entry field to demonstrate how we can receive input from a user in the notification toast and act on it within our application.

NOTE

There is a lot of configuration and code required to implement notification handling. If you would like to open the completed solution and follow along, the code is available on GitHub: **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter08/Complete**.

To get started, open your **MyMediaCollection** solution from the previous chapter or the starter solution for ***Chapter 8*** on GitHub: **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter08/Start**:

1. The first step is to add some configuration to the `Package.appxmanifest` file to enable notification handling in the app. Start by adding two namespace declarations to the `Package` element:

```
xmlns:com="http://schemas.microsoft.com/appx/manifest/com/windows10"
xmlns:desktop="http://schemas.microsoft.com/appx/manifest/desktop/windows10"
```

2. Next, add an `Extensions` section inside the `Application` node, immediately after the `uap:VisualElements` section:

```
<Extensions>
  <desktop:Extension Category="windows.toastNotificationActivation">
    <desktop:ToastNotificationActivation ToastActivatorCLSID="NEW GUID HERE" />
  </desktop:Extension>
  <com:Extension Category="windows.comServer">
    <com:ComServer>
      <com:ExeServer Executable="MyMediaCollection\MyMediaCollection.exe" DisplayName="My Media
        <com:Class Id="SAME NEW GUID HERE" />
      </com:ExeServer>
    </com:ComServer>
  </com:Extension>
</Extensions>
```

   Generate a new **GUID** and replace the two preceding highlighted sections of code with that GUID. It will be unique to your app. You can generate GUIDs in Visual Studio in the menu with **Tools** | **Create GUID**.
3. Next, we're going to create some shared code and helper classes. Your project is not going to compile without errors until we're finished with this section. First, create a new folder in your solution named `Helpers`.

4. Now create a new class named **NotificationShared** in the **Helpers** folder. Start by adding a constant and a struct to this class:

```
public const string scenarioTag = "scenarioId";
public struct Notification
{
    public string Originator;
    public string Action;
    public bool HasInput;
    public string Input;
};
```

The **Notification** struct will represent the data received in an app notification. **scenarioTag** is a constant that will be needed when each notification to send is being constructed.

5. Next, add the following static methods to the **NotificationShared** class. These will be used by the app to notify the UI when notifications are sent or received:

```
public static void CouldNotSendToast()
{
    MainPage.Current.NotifyUser("Could not send toast", InfoBarSeverity.Error);
}
public static void ToastSentSuccessfully()
{
    MainPage.Current.NotifyUser("Toast sent successfully!", InfoBarSeverity.Success);
}
public static void AppLaunchedFromNotification()
{
    MainPage.Current.NotifyUser("App launched from notifications", InfoBarSeverity.Informationa
}
public static void NotificationReceived()
{
    MainPage.Current.NotifyUser("Notification received", InfoBarSeverity.Informational);
}
public static void UnrecognizedToastOriginator()
{
    MainPage.Current.NotifyUser("Unrecognized Toast Originator or Unknown Error", InfoBarSeveri
}
```

MainPage doesn't have a **Current** property, so this code won't compile yet. We'll take care of that soon. If Visual Studio didn't add the necessary **using** statements, make sure these are present in **NotificationShared**:

```
using Microsoft.UI.Xaml.Controls;
using MyMediaCollection.Views;
```

6. Now we are going to create two classes to represent the two types of notifications that the app will send and receive. First, create a new class named **ToastWithAvatar** and start by adding two constants to the class:

```
using Microsoft.Windows.AppNotifications.Builder;
using Microsoft.Windows.AppNotifications;
using MyMediaCollection.Views;
namespace MyMediaCollection.Helpers
{
```

```
    public class ToastWithAvatar
    {
        public const int ScenarioId = 1;
        public const string ScenarioName = "Local Toast with Image";
    }
}
```

7. Next, add a method named **`SendToast`** to the class. This method will construct and show a Windows notification toast containing some text, an avatar image, and a button to display our app:

```
public static bool SendToast()
{
    var appNotification = new AppNotificationBuilder()
        .AddArgument("action", "ToastClick")
        .AddArgument(NotificationShared.scenarioTag, ScenarioId.ToString())
        .SetAppLogoOverride(new System.Uri($"file://{App.GetFullPathToAsset(" Square150x150Logo
        .AddText(ScenarioName)
        .AddText("This is a notification message.")
        .AddButton(new AppNotificationButton("Open App")
            .AddArgument("action", "OpenApp")
            .AddArgument(NotificationShared.scenarioTag, ScenarioId.ToString()))
        .BuildNotification();
    AppNotificationManager.Default.Show(appNotification);
    // If notification is sent, it will have an Id. Success.
    return appNotification.Id != 0;
}
```

8. Now add a **`NotificationReceived`** method, which will be invoked when this type of notification is received from Windows by our app. This method creates a **`Notification`** struct and calls a **`NotificationReceived`** method on **`MainPage`**, which we will create later in this section. We will also create the **`ToForeground`** method to bring our app to the front if it's hidden behind other windows or minimized:

```
public static void NotificationReceived(AppNotificationActivatedEventArgs notificationActivated
{
    var notification = new NotificationShared.Notification
    {
        Originator = ScenarioName,
        Action = notificationActivatedEventArgs.Arguments["action"]
    };
    MainPage.Current.NotificationReceived(notification);
    App.ToForeground();
}
```

9. The **`ToastWithText`** class will be similar to **`ToastWithAvatar`**, but it will add a call to **`AddTextBox`** in **`AppNotificationBuilder`** to create the input field in the Windows toast. It also adds the result of that user input to the **`Notification`** class created in **`NotificationReceived`**. To view the full code for this class, check out the completed solution on GitHub: **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter08/Complete/MyMediaCollection/Helpers/ToastWithText.cs**.
10. Now it's time to create the **`NotificationManager`** class. This class will do exactly that – manage notifications. It will initialize and unregister

notification receiving. It will do the actual sending and receiving of notifications. Create the **`NotificationManager`** class in the **`Helpers`** folder and start by adding the constructor and finalization code:

```
using Microsoft.Windows.AppNotifications;
using System;
using System.Collections.Generic;
namespace MyMediaCollection.Helpers
{
    internal class NotificationManager
    {
        private bool isRegistered;
        private Dictionary<int, Action<AppNotificationActivatedEventArgs>> notificationHandlers
        public NotificationManager()
        {
            isRegistered = false;
            notificationHandlers = new Dictionary<int, Action<AppNotificationActivatedEventArgs
            {
                { ToastWithAvatar.ScenarioId, ToastWithAvatar.NotificationReceived },
                { ToastWithText.ScenarioId, ToastWithText.NotificationReceived }
            };
        }
        ~NotificationManager()
        {
            Unregister();
        }
        public void Unregister()
        {
            if (isRegistered)
            {
                AppNotificationManager.Default.Unregister();
                isRegistered = false;
            }
        }
    }
}
```

11. Next, add the **`Init`** method that we'll call from the **`App`** class:

```
public void Init()
{
    AppNotificationManager notificationManager = AppNotificationManager.Default;
    // Add handler before calling Register.
    notificationManager.NotificationInvoked += OnNotificationInvoked;
    notificationManager.Register();
    isRegistered = true;
}
```

12. **`OnNotificationInvoked`** is hooked up in the **`Init`** method. This will fire when notifications are received by the app. It makes different calls to **`NotificationShared`**, depending on whether the notification is recognized or not:

```
public void OnNotificationInvoked(object sender, AppNotificationActivatedEventArgs notification
{
    NotificationShared.NotificationReceived();
    if (!DispatchNotification(notificationActivatedEventArgs))
    {
        NotificationShared.UnrecognizedToastOriginator();
```

```
    }
}
```

NOTE

If you have any unhandled exceptions in your code that process incoming notifications, they will also trigger this call to **`NotificationShared.UnrecognizedToastOriginator`**.

13. Finally, create the **`ProcessLaunchActivationArgs`** and **`DispatchNotification`** methods in **`NotificationManager`**:

```
public void ProcessLaunchActivationArgs(AppNotificationActivatedEventArgs notificationActivated
{
    DispatchNotification(notificationActivatedEventArgs);
    NotificationShared.AppLaunchedFromNotification();
}
private bool DispatchNotification(AppNotificationActivatedEventArgs notificationActivatedEventA
{
    var scenarioId = notificationActivatedEventArgs.Arguments[NotificationShared.scenarioTag];
    if (scenarioId.Length != 0)
    {
        try
        {
            notificationHandlers[int.Parse(scenarioId)](notificationActivatedEventArgs);
            return true;
        }
        catch
        {
            // No matching handler
            return false;
        }
    }
    else
    {
        // No scenarioId provided
        return false;
    }
}
```

14. Now let's add the code to **`App.xaml.cs`** to initialize **`NotificationManager`** and handle some of the common calls. Let's start by adding the **`using`** statements that we'll need for the new code:

```
using Microsoft.Windows.AppLifecycle;
using Microsoft.Windows.AppNotifications;
using MyMediaCollection.Helpers;
using System.Runtime.InteropServices;
using WinRT.Interop;
```

15. Next, add a private **`notificationManager`** object, add **`DllImport`** to help bring the window to the foreground, and make **`m_window`** static:

```
[DllImport("user32.dll", SetLastError = true)]
static extern void SwitchToThisWindow(IntPtr hWnd, bool turnOn);
private NotificationManager notificationManager;
private static Window m_window;
```

NOTE

Be careful when choosing which Win32 APIs to use in your production WinUI applications. The **`SwitchToThisWindow`** *API is documented as "not suitable for general use," but it works for our purposes in a sample app. There are other APIs you can explore, including* **`ShowWindow`***:* **https://learn.microsoft.com/windows/win32/api/winuser/nf-winuser-showwindow**.

16. Next, add the following code to **`OnLaunched`** right before calling **`m_window.Activate`**. This gets the arguments passed to the app from the Windows notification:

```
var currentInstance = AppInstance.GetCurrent();
if (currentInstance.IsCurrent)
{
    AppActivationArguments activationArgs = currentInstance.GetActivatedEventArgs();
    if (activationArgs != null)
    {
        ExtendedActivationKind extendedKind = activationArgs.Kind;
        if (extendedKind == ExtendedActivationKind.AppNotification)
        {
            var notificationActivatedEventArgs = (AppNotificationActivatedEventArgs)activationA
            notificationManager.ProcessLaunchActivationArgs(notificationActivatedEventArgs);
        }
    }
}
```

17. Next, add some code to the **`App`** constructor to initialize **`NotificationManager`** and handle the **`AppDomain.CurrentDomain.ProcessExit`** event to unregister the manager when the app closes:

```
public App()
{
    this.InitializeComponent();
    notificationManager = new NotificationManager();
    notificationManager.Init();
    AppDomain.CurrentDomain.ProcessExit += CurrentDomain_ProcessExit;
}
private void CurrentDomain_ProcessExit(object sender, EventArgs e)
{
    notificationManager.Unregister();
}
```

18. The final items to be added to the **`App`** class are three static helper methods to get some application-related paths and the **`ToForeground`** method to bring the app to the front when it's hidden or minimized:

```
public static void ToForeground()
{
    if (m_window != null)
    {
        IntPtr handle = WindowNative.GetWindowHandle(m_window);
        if (handle != IntPtr.Zero)
        {
            SwitchToThisWindow(handle, true);
```

```
        }
    }
}
public static string GetFullPathToExe()
{
    var path = AppDomain.CurrentDomain.BaseDirectory;
    var pos = path.LastIndexOf("\\");
    return path.Substring(0, pos);
}
public static string GetFullPathToAsset(string assetName)
{
    return $"{GetFullPathToExe()}\\Assets\\{assetName}";
}
```

19. Now let's work on **`MainPage`**. Start in **`MainPage.xaml`**. We're going to add two buttons to send notifications and an **`InfoBar`** control to display messages at the bottom of the page when notifications are sent or received. Add another **`RowDefinition`** to the outermost **`Grid`** control:

```
<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*"/>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
```

20. Add two buttons to the beginning of **`StackPanel`** that contains the existing **`Button`** controls:

```
<StackPanel HorizontalAlignment="Right"
            Orientation="Horizontal">
    <Button Command="{x:Bind ViewModel.SendToastCommand}"
            Content="Send Notification"
            Margin="8,8,0,8"/>
    <Button Command="{x:Bind ViewModel.SendToastWithTextCommand}"
            Content="Send Notification with Text"
            Margin="8,8,0,8"/>
...
</StackPanel>
```

21. Add an **`InfoBar`** control just before the closing tag for the outer **`Grid`**:

```
<InfoBar x:Name="notifyInfoBar" Grid.Row="3"/>
```

22. Next, open the **`MainPage.xaml.cs`** file, in which we need to add some code to handle the incoming notifications. The first thing we'll do is add a **`using`** statement for **`MyMediaCollection.Helpers`**.
23. Next, add the code to expose the current instance of **`MainPage`** so the notifications can be routed properly:

```
public static MainPage Current;
public MainPage()
{
    ViewModel = App.HostContainer.Services.GetService<MainViewModel>();
    this.InitializeComponent();
    Current = this;
    Loaded += MainPage_Loaded;
}
```

24. Next, add some code to update **`InfoBar`** whenever notifications are sent or received. This is the code that will be called by the methods in the **`NotificationShared`** class:

```
public void NotifyUser(string message, InfoBarSeverity severity, bool isOpen = true)
{
    if (DispatcherQueue.HasThreadAccess)
    {
        UpdateStatus(message, severity, isOpen);
    }
    else
    {
        DispatcherQueue.TryEnqueue(() =>
        {
            UpdateStatus(message, severity, isOpen);
        });
    }
}
private void UpdateStatus(string message, InfoBarSeverity severity, bool isOpen)
{
    notifyInfoBar.Message = message;
    notifyInfoBar.IsOpen = isOpen;
    notifyInfoBar.Severity = severity;
}
```

    The **`DispatcherQueue`** methods check whether the code has access to the UI thread. If not, **`TryEnqueue`** is used to queue the work to be performed when the UI thread is available. Otherwise, errors will be encountered when accessing UI elements from a background thread.

25. Create a **`NotificationReceived`** method to handle incoming notification information. This method parses through the incoming data and builds a message string to display:

```
public void NotificationReceived(NotificationShared.Notification notification)
{
    var text = $"{notification.Originator}; Action: {notification.Action}";
    if (notification.HasInput)
    {
        if (string.IsNullOrWhiteSpace(notification.Input))
            text += "; No input received";
        else
            text += $"; Input received: {notification.Input}";
    }
    if (DispatcherQueue.HasThreadAccess)
        DisplayMessageDialog(text);
    else
    {
        DispatcherQueue.TryEnqueue(() =>
        {
            DisplayMessageDialog(text);
        });
    }
}
```

    The final code to add to **`MainPage`** is a simple method to display **`ContentDialog`** with the notification data:

```
private void DisplayMessageDialog(string message)
{
```

```
    ContentDialog notifyDialog = new()
    {
        XamlRoot = this.XamlRoot,
        Title = "Notification received",
        Content = message,
        CloseButtonText = "Ok"
    };
    notifyDialog.ShowAsync();
}
```

`MainViewModel` is the final class to be updated. We need to create two command methods for the new buttons to invoke when sending app notifications. Create two methods named **`SendToast`** and **`SendToastWithText`**:

```
[RelayCommand]
private void SendToast()
{
    if (ToastWithAvatar.SendToast())
        NotificationShared.ToastSentSuccessfully();
    else
        NotificationShared.CouldNotSendToast();
}
[RelayCommand]
private void SendToastWithText()
{
    if (ToastWithText.SendToast())
        NotificationShared.ToastSentSuccessfully();
    else
        NotificationShared.CouldNotSendToast();
}
```

Don't forget to add **`using MyMediaCollection.Helpers;`** to the list of using statements in **`MainViewModel`**.

We're ready to run the app and test the notifications. Start debugging and click the **Send Notification** button. You should see a toast appear in the lower-right portion of the main screen:

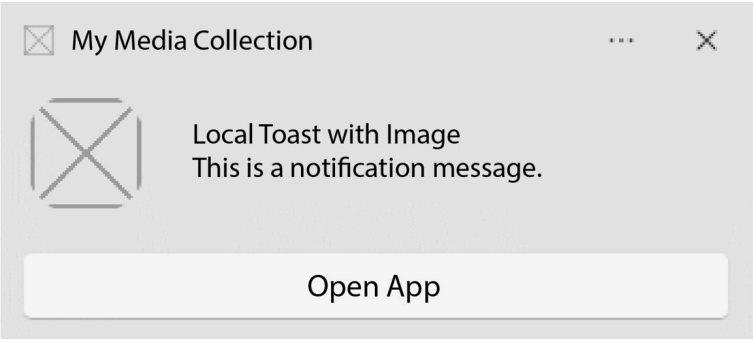

Figure 8.1 – A Windows toast notification

Bring another window in front of **My Media Collection** and then click **Open App** on the toast. The app should be brought back to the front of the screen and **ContentDialog** will display a message with information about the notification received:

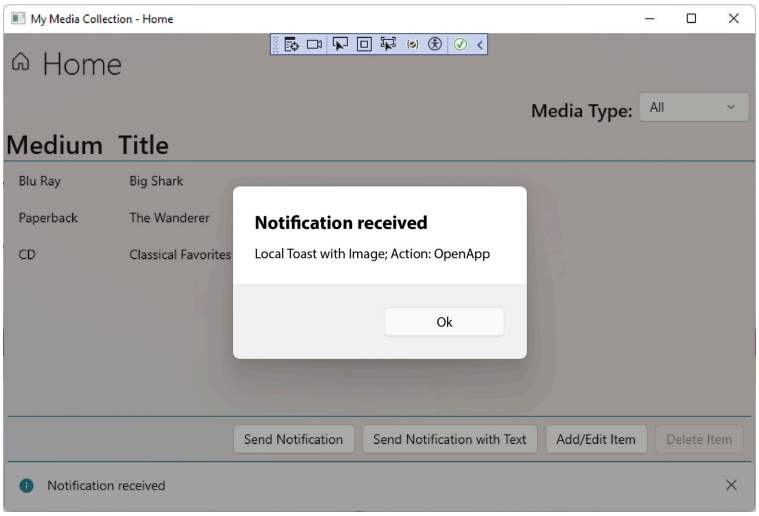

Figure 8.2 – Receiving an app notification

NOTE

If you click one of the buttons to send a notification multiple times without acknowledging the toast window, the toasts will not stack. The subsequent toasts will go straight to Windows' **Notifications Center**. *Once they go there, it's not possible to use interactive fields such as a text box. In addition, if the user has Do Not Disturb or Focus mode enabled, all notifications will be suppressed and sent directly to* **Notifications Center**.

Now click the **Send Notification with Text** button. When this toast appears, it will have a text box where you can **Enter a reply**:

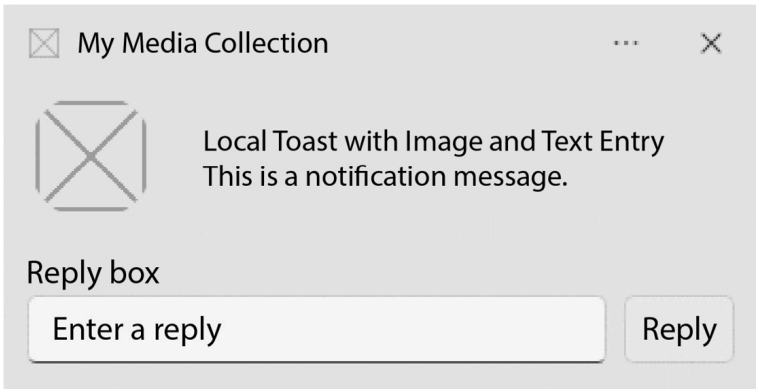

Figure 8.3 – Displaying a toast window with a text box

Type `Hello world` and click the **Reply** button. Now, when **My Media Collection** displays **ContentDialog**, it will include the reply that was entered in the toast window:

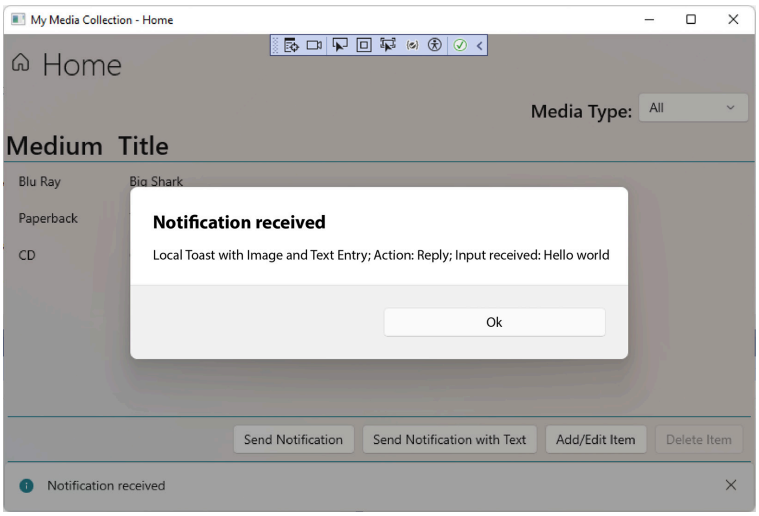

Figure 8.4 – Displaying the reply text from the toast window in our app

Now you're ready to start building notifications into your own WinUI applications. Let's wrap up the chapter and discuss what we've learned.

Summary

In this chapter, we learned about the types of Windows notifications available to WinUI developers in the Windows App SDK. We discussed how notifications can be used to save Windows resources, reduce the need for timers, and prompt users to act. We explored how to configure raw push notifications and added a local app notification to the **My Media Collection** sample app. You should now feel prepared to add any of these types of notifications to your own WinUI applications.

In the next chapter, we'll explore the **Windows Community Toolkit (WCT)** and learn how, together with the .NET Community Toolkit, you can save development time by leveraging existing helpers, styles, and controls.

Questions

1. What type of Windows notification can be used to initiate a data sync from the cloud?
2. Which type of notification doesn't rely on WNS?
3. Where do you register your app before configuring notification services in Azure?
4. Which Windows App SDK namespace contains the objects for working with app notifications?
5. Which class has methods to register and unregister your app for handling app notifications?
6. Which property can be set if you would like notifications from your app to disappear after a system reboot?
7. Can notifications from WNS prompt the user with a toast notification?